THE UNIVERSITY OF MELBOURNE

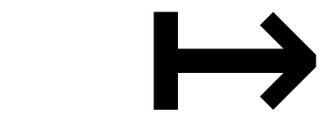
Separation Logic for Automated Verification

A Partial Overview

Christine Rizkallah


christine.rizkallah@unimelb.edu.au


http://people.eng.unimelb.edu.au/rizkallahc



Introduction to Separation Logic

Hoare Reasoning

THE UNIVERSITY OF MELBOURNE

Hoare Triple: {P} prog {Q}

Hoare Reasoning

Hoare Triple: {P} prog {Q}

$$\text{CONSEQ}\ \frac{P \Rightarrow P' \quad \{P'\}\ \text{prog}\ \{Q'\} \quad Q' \Rightarrow Q}{\{P\}\ \text{prog}\ \{Q\}}$$

Hoare Reasoning

THE UNIVERSITY OF MELBOURNE

Hoare Triple: {P} prog {Q}

$$\text{CONSEQ}\ \frac{P \Rightarrow P' \quad \{P'\}\ \text{prog}\ \{Q'\} \quad Q' \Rightarrow Q}{\{P\}\ \text{prog}\ \{Q\}}$$

$$\text{ASSIGN}\ \{Q[e/x]\}\ x := e\ \{Q\}$$

Hoare Reasoning

Hoare Triple: {P} prog {Q}

$$\text{CONSEQ}\ \frac{P \Rightarrow P' \quad \{P'\}\ \text{prog}\ \{Q'\} \quad Q' \Rightarrow Q}{\{P\}\ \text{prog}\ \{Q\}}$$

ASSIGN $\{Q[e/x]\}\ x := e\ \{Q\}$

For **local variables** x and y:

$$\frac{\{2 = 2 \wedge y = 3\}\ x := 2\ \{x = 2 \wedge y = 3\}\ \text{(by ASSIGN)}}{\{x = 3 \wedge y = 3\}\ x := 2\ \{x = 2 \wedge y = 3\}\ \text{(by CONSEQ)}}$$

Hoare Reasoning

Hoare Triple: {P} prog {Q}

$$\text{CONSEQ}\ \frac{P \Rightarrow P' \quad \{P'\}\ \text{prog}\ \{Q'\} \quad Q' \Rightarrow Q}{\{P\}\ \text{prog}\ \{Q\}}$$

$$\text{ASSIGN}\ \{Q[e/x]\}\ x := e\ \{Q\}$$

For **local variables** x and y:

$$\frac{\{2 = 2 \wedge y = 3\}\ x := 2\ \{x = 2 \wedge y = 3\}\ \ (\text{by ASSIGN})}{\{x = 3 \wedge y = 3\}\ x := 2\ \{x = 2 \wedge y = 3\}\ \ (\text{by CONSEQ})}$$

(ASSIGN is) valid (only) because x and y cannot alias.

THE UNIVERSITY OF MELBOURNE

How do we extend Hoare logic to handle programs with pointers?

How do we extend Hoare logic to handle programs with pointers?

The answer is **Separation Logic**

How do we extend Hoare logic to handle programs with pointers?

The answer is **Separation Logic**

But before lets see how can we come up with a notion of **memory (aka heap)**,
the memory that pointers are pointing to

Heap Reasoning

What if x and y are pointers into the heap?

*x := 2

Heap Reasoning

What if x and y are pointers into the heap?

*x := 2

Maps-To Assertion: p ↦ e

"the heap location identified by p holds the value e"

Heap Reasoning

What if x and y are pointers into the heap?

*x := 2

Maps-To Assertion: p ↦ e

"the heap location identified by p holds the value e"

Is this Hoare triple valid?

{x ↦ 3 ∧ y ↦ 3} *x := 2 {x ↦ 2 ∧ y ↦ 3}

Heap Reasoning

What if x and y are pointers into the heap?

*x := 2

Maps-To Assertion: p ↦ e

"the heap location identified by p holds the value e"

Is this Hoare triple valid?

{x ↦ 3 ∧ y ↦ 3} *x := 2 {x ↦ 2 ∧ y ↦ 3}

No.

Heap Reasoning

What if x and y are pointers into the heap?

*x := 2

Maps-To Assertion: p ↦ e

"the heap location identified by p holds the value e"

Is this Hoare triple valid?

{x ↦ 3 ∧ y ↦ 3} *x := 2 {x ↦ 2 ∧ y ↦ 3}

No.

```
void foo(int *x){
    int *y := x;
    *x := 3;
    *x := 2;
}
```

Heap Reasoning

What if x and y are pointers into the heap?

*x := 2

Maps-To Assertion: p ↦ e

"the heap location identified by p holds the value e"

Is this Hoare triple valid?

{x ↦ 3 ∧ y ↦ 3} *x := 2 {x ↦ 2 ∧ y ↦ 3}

No.

```
void foo(int *x){
    int *y := x;
    *x := 3;
    *x := 2;
}
```

It does't hold when x and y **alias** each other.

Global Reasoning

We need a way to reason **locally** about statements like

*x := 2

Hoare predicates like P and Q in {P} prog {Q} talk **globally:** they hold, or not, for the entire state.

Global Reasoning

We need a way to reason **locally** about statements like

*x := 2

Hoare predicates like P and Q in {P} prog {Q} talk **globally:** they hold, or not, for the entire state.

loc_0	val_0
loc_0	val_0
…	…
…	…

Global Reasoning

We need a way to reason **locally** about statements like

*x := 2

Hoare predicates like P and Q in {P} prog {Q} talk **globally:** they hold, or not, for the entire state.

P

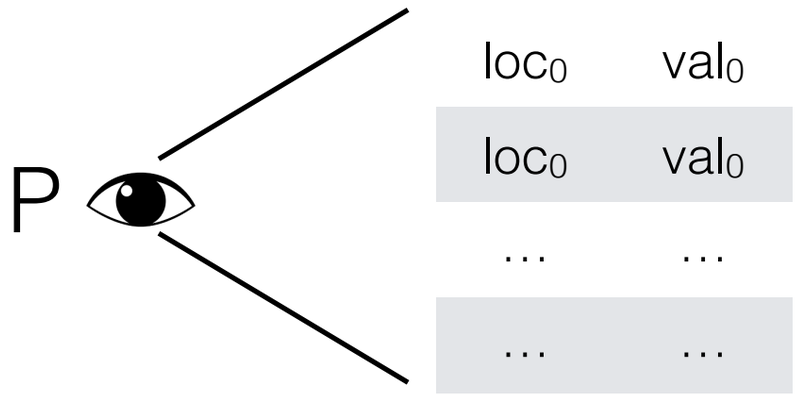
loc_0	val_0
loc_0	val_0
…	…
…	…

Global Reasoning

We need a way to reason **locally** about statements like

*x := 2

Hoare predicates like P and Q in {P} prog {Q} talk **globally:** they hold, or not, for the entire state.



Local Reasoning

Idea: have predicates P characterise the part of the state that they talk about.

Local Reasoning

Idea: have predicates P characterise the part of the state that they talk about.

$loc_i \mapsto val_i$

Local Reasoning

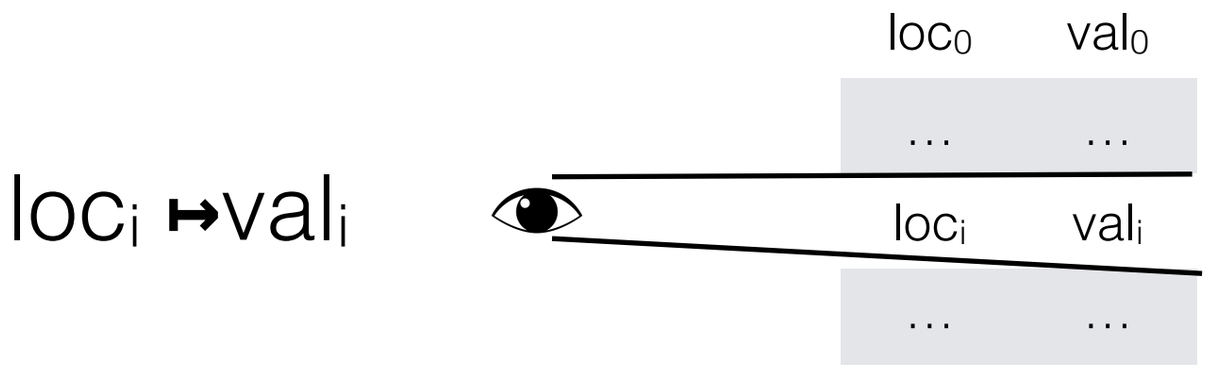
Idea: have predicates P characterise the part of the state that they talk about.

$loc_i \mapsto val_i$

loc_0	val_0
…	…
loc_i	val_i
…	…

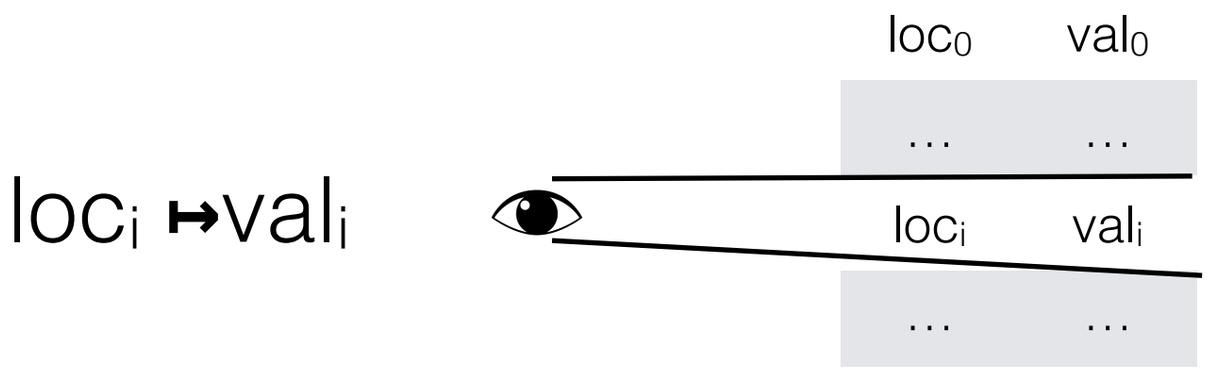
Local Reasoning

Idea: have predicates P characterise the part of the state that they talk about.



Local Reasoning

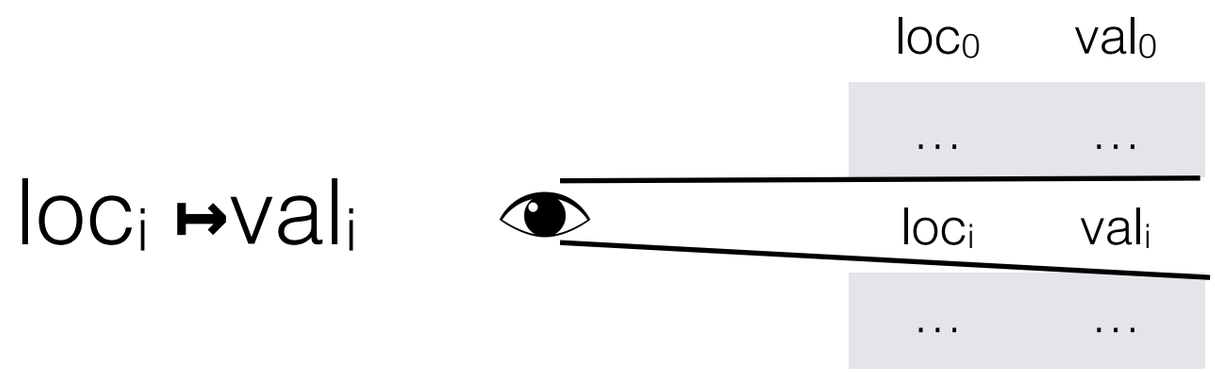
Idea: have predicates P characterise the part of the state that they talk about.



This part is called the predicate's **domain** (aka footprint), written dom(P)

Local Reasoning

Idea: have predicates P characterise the part of the state that they talk about.



This part is called the predicate's **domain** (aka footprint), written dom(P)

$$dom(p \mapsto e) = \{p\}$$

Local Reasoning

THE UNIVERSITY OF MELBOURNE

A valid triple:

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

Local Reasoning

THE UNIVERSITY OF MELBOURNE

A valid triple:

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

Any pre-state satisfying $x \mapsto 3$

loc_0	val_0
…	…
x	3
…	…

Local Reasoning

A valid triple:

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

Any pre-state satisfying $x \mapsto 3$

loc_0	val_0
…	…
x	3
…	…

Post-State

loc_0	val_0
…	…
x	2
…	…

Local Reasoning

A valid triple:

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

Any pre-state satisfying $x \mapsto 3$

loc_0	val_0
…	…
x	3
…	…

Post-State

loc_0	val_0
…	…
x	2
…	…

Satisfies $x \mapsto 2$

Local Reasoning

A valid triple:

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

Any pre-state satisfying $x \mapsto 3$

loc_0	val_0
…	…
x	3
…	…

Post-State

loc_0	val_0
…	…
x	2
…	…

Satisfies $x \mapsto 2$

How do we talk about **other** parts of the heap?

Compositional Reasoning

THE UNIVERSITY OF MELBOURNE

Separating Conjunction: $P \star Q$

"P holds for its part of the heap and Q holds for its part and, moreover, **those two parts are separate**."

Compositional Reasoning

THE UNIVERSITY OF MELBOURNE

Separating Conjunction: $P \star Q$

"P holds for its part of the heap and Q holds for its part and, moreover, **those two parts are separate**."

$$x \mapsto 3 \star y \mapsto 3$$

implies that $x \neq y$, i.e. **x and y do not alias**

Compositional Reasoning

THE UNIVERSITY OF MELBOURNE

Separating Conjunction: $P \star Q$

"P holds for its part of the heap and Q holds for its part and, moreover, **those two parts are separate**."

$$x \mapsto 3 \star y \mapsto 3$$

implies that $x \neq y$, i.e. **x and y do not alias**

Is this triple valid?

$$\{x \mapsto 3 \star y \mapsto 3\}\ *x := 2\ \{x \mapsto 2 \star y \mapsto 3\}$$

Compositional Reasoning

THE UNIVERSITY OF MELBOURNE

Separating Conjunction: $P \star Q$

"P holds for its part of the heap and Q holds for its part and, moreover, **those two parts are separate**."

$$x \mapsto 3 \star y \mapsto 3$$

implies that $x \neq y$, i.e. **x and y do not alias**

Is this triple valid?

$$\{x \mapsto 3 \star y \mapsto 3\}\ {*x} := 2\ \{x \mapsto 2 \star y \mapsto 3\}$$

Yes.

The Frame Rule

THE UNIVERSITY OF MELBOURNE

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

The Frame Rule

THE UNIVERSITY OF MELBOURNE

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

The Frame Rule

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Allows us to derive

$\{x \mapsto 3 \star y \mapsto 3\}\ *x := 2\ \{x \mapsto 2 \star y \mapsto 3\}$

from

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

The Frame Rule

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Allows us to derive

$\{x \mapsto 3 \star y \mapsto 3\}$ *x := 2 $\{x \mapsto 2 \star y \mapsto 3\}$

from

$\{x \mapsto 3\}$ *x := 2 $\{x \mapsto 2\}$

modv(prog): the set of local variables modified by prog

The Frame Rule

THE UNIVERSITY OF MELBOURNE

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Allows us to derive

$\{x \mapsto 3 \star y \mapsto 3\}\ *x := 2\ \{x \mapsto 2 \star y \mapsto 3\}$

from

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

modv(prog): the set of local variables modified by prog

fv(R): the set of free variables mentioned by R

The Frame Rule

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Allows us to derive

$\{x \mapsto 3 \star y \mapsto 3\}$ *x := 2 $\{x \mapsto 2 \star y \mapsto 3\}$

from

$\{x \mapsto 3\}$ *x := 2 $\{x \mapsto 2\}$

modv(prog): the set of local variables modified by prog

fv(R): the set of free variables mentioned by R

modv(*x := 2) = {}

The Frame Rule

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad modv(prog) \cap fv(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Allows us to derive

$\{x \mapsto 3 \star y \mapsto 3\}\ *x := 2\ \{x \mapsto 2 \star y \mapsto 3\}$

from

$\{x \mapsto 3\}\ *x := 2\ \{x \mapsto 2\}$

modv(prog): the set of local variables modified by prog

fv(R): the set of free variables mentioned by R

$modv(*x := 2) = \{\}$ $\qquad$ $fv(y \mapsto 3) = \{y\}$

The Side Condition

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad \mathrm{modv}(prog) \cap \mathrm{fv}(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

$$\mathrm{modv}(*x := 2;\ y := x) = \{y\}$$
$$\mathrm{fv}(y \mapsto 3) = \{y\}$$

Side condition doesn't hold.

The Side Condition

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad \mathrm{modv}(prog) \cap \mathrm{fv}(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Suppose we had:

$\{x \mapsto 3\}$ *x := 2; y := x $\{x \mapsto 2\}$

$\mathrm{modv}(*x := 2; y := x) = \{y\}$

$\mathrm{fv}(y \mapsto 3) = \{y\}$

Side condition doesn't hold.

The Side Condition

$$\text{FRAME}\ \frac{\{P\}\ prog\ \{Q\} \qquad \mathrm{modv}(prog) \cap \mathrm{fv}(R) = \{\}}{\{P \star R\}\ prog\ \{Q \star R\}}$$

Suppose we had:

$$\{x \mapsto 3\}\ {*x} := 2;\ y := x\ \{x \mapsto 2\}$$

Clearly, it is **not** the case that:

$$\{x \mapsto 3 \star y \mapsto 3\}\ {*x} := 2;\ y := x\ \{x \mapsto 2 \star y \mapsto 3\}$$

$\mathrm{modv}(*x := 2;\ y := x) = \{y\}$

$\mathrm{fv}(y \mapsto 3) = \{y\}$

Side condition doesn't hold.

Validity of Triples

THE UNIVERSITY OF MELBOURNE

Meaning of {P} prog {Q} **in Separation Logic**

Validity of Triples

THE UNIVERSITY OF MELBOURNE

Meaning of {P} prog {Q} **in Separation Logic**

Partial Correctness: If P holds initially then whenever prog terminates successfully, then Q holds and, moreover, **prog never fails.**

Memory Safety

THE UNIVERSITY OF MELBOURNE

Example: {x = `NULL`} y := *x {true}

Does this triple hold?

Memory Safety

Example: {x = `NULL`} y := *x {true}

Does this triple hold?

No.

Memory Safety

THE UNIVERSITY OF MELBOURNE

Example: {x = `NULL`} y := *x {true}

Does this triple hold?

No.

In a language semantics in which all memory-unsafe accesses cause failure,
separation logic guarantees memory safety.

Memory Safety

Example: {x = `NULL`} y := *x {true}

Does this triple hold?

No.

In a language semantics in which all memory-unsafe accesses cause failure,
separation logic guarantees memory safety.

e.g. accessing an uninitialised pointer.

```
int *p;
*p = 2;
```

Expressions, Commands

THE UNIVERSITY OF MELBOURNE

Expressions: $e ::= x \mid n \mid \ldots$

May mention (local) variables but not the heap.

Are evaluated against a store s :: Var → Int

$[\![e]\!]_s$ :: Int

Expressions, Commands

THE UNIVERSITY OF MELBOURNE

Expressions: $e ::= x \mid n \mid \ldots$

May mention (local) variables but not the heap.

Are evaluated against a store s :: Var → Int

$[\![e]\!]_s$ :: Int

Commands: $c ::= c_1 \,;\, c_2 \mid$ **if** e **then** c_t **else** c_f
$\mid$ **while** e **do** $c \mid x := e$
$\mid x := {}^*e \mid {}^*e_1 := e_2$

Expressions, Commands

THE UNIVERSITY OF MELBOURNE

Expressions: $e ::= x \mid n \mid \ldots$

May mention (local) variables but not the heap.

Are evaluated against a store s :: Var → Int

$[\![e]\!]_s$ :: Int

Commands: $c ::= c_1 \,;\, c_2 \mid$ **if** e **then** c_t **else** c_f
$\mid$ **while** e **do** $c \mid x := e$
$\mid x := {}^*e \mid {}^*e_1 := e_2$

Expressions, Commands

THE UNIVERSITY OF MELBOURNE

Expressions: $e ::= x \mid n \mid \ldots$

May mention (local) variables but not the heap.

Are evaluated against a store s :: Var → Int

$[\![e]\!]_s$ :: Int

Commands: $c ::= c_1 \,;\, c_2 \mid$ **if** e **then** c_t **else** c_f
$\mid$ **while** e **do** $c \mid x := e$
$\mid x := {}^*e \mid {}^*e_1 := e_2$

Note: heap- and store-update are syntactically distinct.

Heap Read Rule (Simplified)

THE UNIVERSITY OF MELBOURNE

$$\frac{x \notin \text{vars}(e)}{\{e \mapsto v\}\ x := {}^*e\ \{x = v \wedge e \mapsto v\}}$$

Heap Read Rule (Simplified)

THE UNIVERSITY OF MELBOURNE

$$\frac{x \notin \mathrm{vars}(e)}{\{e \mapsto v\}\ x := {*e}\ \{x = v \wedge e \mapsto v\}}$$

$$x := {*(y+1)}$$

Heap Read Rule (Simplified)

THE UNIVERSITY OF MELBOURNE

$$\frac{x \notin \text{vars}(e)}{\{e \mapsto v\}\ x := {}^*e\ \{x = v \wedge e \mapsto v\}}$$

$s(y) = 19$

h

…	…
4	2
…	…
20	3
…	…

x := *(y+1)

Heap Read Rule (Simplified)

THE UNIVERSITY OF MELBOURNE

$$\frac{x \notin \text{vars}(e)}{\{e \mapsto v\}\ x := {}^*e\ \{x = v \wedge e \mapsto v\}}$$

$s(y) = 19$

h

…	…
4	2
…	…
20	3
…	…

$x := {}^*(y+1)$

$y+1 \mapsto 3$ holds

Heap Read Rule (Simplified)

THE UNIVERSITY OF MELBOURNE

$$\frac{x \notin \text{vars}(e)}{\{e \mapsto v\}\ x := {}^*e\ \{x = v \wedge e \mapsto v\}}$$

$s(y) = 19$

h

…	…
4	2
…	…
20	3
…	…

$x := {}^*(y+1)$

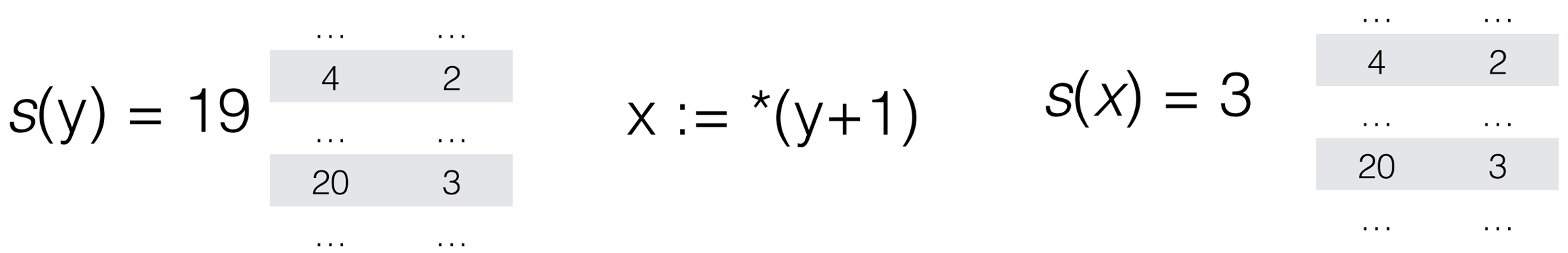
$s(x) = 3$

…	…
4	2
…	…
20	3
…	…

$y+1 \mapsto 3$ holds

Heap Read Rule (Simplified)

$$\frac{x \notin \text{vars}(e)}{\{e \mapsto v\}\ x := {}^*e\ \{x = v \wedge e \mapsto v\}}$$



y+1 ↦ 3 holds

NOTE: We need a more general rule for when x ∈ vars(e) but that is beyond the scope of today's lecture

Assignment Rules

THE UNIVERSITY OF MELBOURNE

HEAPW $\quad \{e_1 \mapsto \_\}\ {*e_1} := e_2\ \{e_1 \mapsto e_2\}$

ASSIGN $\quad \{x \doteq n\}\ x := e\ \{x \doteq e[n/x]\}$

ASSIGN' $\quad \dfrac{x \notin \mathrm{vars}(e)}{\{\mathbf{emp}\}\ x := e\ \{x \doteq e\}}$

Frame Rule

$$\text{modv}(x := e) = \{x\}$$
$$\text{modv}(x := *e) = \{x\}$$
$$\text{modv}(*e_1 := e_2) = \{\}$$
$$\text{modv}(\textbf{while } e \textbf{ do } c) = \text{modv}(c)$$
$$\text{modv}(\textbf{if } e \textbf{ then } c_t \textbf{ else } c_f) = \text{modv}(c_t) \cup \text{modv}(c_f)$$
$$\text{modv}(c_1 ; c_2) = \text{modv}(c_1) \cup \text{modv}(c_2)$$

$$\text{FRAME}\ \frac{\{P\}\ c\ \{Q\} \qquad \text{modv}(c) \cap \text{fv}(R) = \{\}}{\{P \star R\}\ c\ \{Q \star R\}}$$

Example Proof

THE UNIVERSITY OF MELBOURNE

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$u := *y;$$
$$*x := u;$$
$$*y := t;$$
$$\{x \mapsto v_y \star y \mapsto v_x\}$$

Example Proof

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$u := *y;$$
$$*x := u;$$
$$*y := t;$$
$$\{x \mapsto v_y \star y \mapsto v_x\}$$

Apply the Hoare sequencing rule

Example Proof

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$*x := u;$$
$$*y := t;$$
$$\{x \mapsto v_y \star y \mapsto v_x\}$$

Example Proof

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$*x := u;$$
$$*y := t;$$
$$\{x \mapsto v_y \star y \mapsto v_x\}$$

Example Proof

Example Proof

$$\{x \mapsto v_x\}$$
$$t := *x;$$
$$\{x \mapsto v_x \wedge t = v_x\}$$

Example Proof

$$\{x \mapsto v_x\}$$
$$t := *x;$$
$$\{x \mapsto v_x \wedge t = v_x\}$$

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$

Example Proof Outline

THE UNIVERSITY OF MELBOURNE

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$

Example Proof Outline

THE UNIVERSITY OF MELBOURNE

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$

Example Proof Outline

THE UNIVERSITY OF MELBOURNE

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$

Example Proof Outline

THE UNIVERSITY OF MELBOURNE

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*x := u;$$

Example Proof Outline

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*x := u;$$
$$\{(x \mapsto u \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$

Example Proof Outline

THE UNIVERSITY OF MELBOURNE

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*x := u;$$
$$\{(x \mapsto u \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*y := t;$$

Example Proof Outline

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*x := u;$$
$$\{(x \mapsto u \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*y := t;$$
$$\{(x \mapsto u \wedge t = v_x) \star (y \mapsto t \wedge u = v_y)\}$$

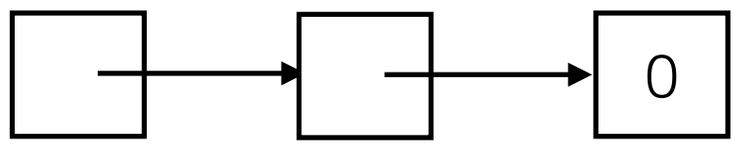
Example Proof Outline

$$\{x \mapsto v_x \star y \mapsto v_y\}$$
$$t := *x;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star y \mapsto v_y\}$$
$$u := *y;$$
$$\{(x \mapsto v_x \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*x := u;$$
$$\{(x \mapsto u \wedge t = v_x) \star (y \mapsto v_y \wedge u = v_y)\}$$
$$*y := t;$$
$$\{x \mapsto v_y \star y \mapsto v_x\}$$

Inductive Predicates

Heap assertions $e_1 \mapsto e_2$ and **emp** talk only about bounded parts of the heap (of size 1 and 0 resp.)

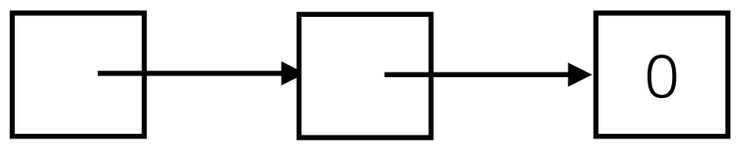
How to talk about unbounded heap data structures?



Inductive Predicates

Heap assertions $e_1 \mapsto e_2$ and **emp** talk only about bounded parts of the heap (of size 1 and 0 resp.)

How to talk about unbounded heap data structures?



$$\text{list}(e) = (e = 0 \wedge \textbf{emp}) \vee (\exists e'.\ e \mapsto e' \star \text{list}(e'))$$

Example: Tail

Example: Tail

$$\{x \neq 0 \wedge \text{list}(x)\}$$
$$t := *x;$$
$$\{\text{list}(t)\}$$

Example: Tail

$$\{x \neq 0 \wedge \text{list}(x)\}$$
$$t := *x;$$
$$\{\text{list}(t)\}$$

Unfold the inductive predicate:

$$\{x \neq 0 \wedge (\exists e'.\ x \mapsto e' \star \text{list}(e'))\}$$
$$t := *x;$$
$$\{\text{list}(t)\}$$

Example: Tail

$$\{x \neq 0 \wedge \text{list}(x)\}$$
$$t := *x;$$
$$\{\text{list}(t)\}$$

Unfold the inductive predicate:

$$\{x \neq 0 \wedge x \mapsto e' \star \text{list}(e')\}$$
$$t := *x;$$
$$\{\text{list}(t)\}$$

Example: Tail

THE UNIVERSITY OF MELBOURNE

$$\{x \neq 0 \wedge x \mapsto e' \star \mathrm{list}(e')\}$$

t := *x;

{list(t)}

Example: Tail

$$\{x \neq 0 \wedge x \mapsto e' \star \text{list}(e')\}$$
$$t := *x;$$
$$\{\text{list}(t)\}$$

$$\{x \neq 0 \wedge x \mapsto e'\}$$
$$t := *x;$$
$$\{x \neq 0 \wedge x \mapsto e' \wedge t = e'\}$$

Example: Tail

$$\{x \neq 0 \wedge x \mapsto e' \star \mathrm{list}(e')\}$$
$$t := *x;$$
$$\{\mathrm{list}(t)\}$$

$$\{x \neq 0 \wedge x \mapsto e'\}$$
$$t := *x;$$
$$\{x \neq 0 \wedge x \mapsto e' \wedge t = e'\}$$

$$\{x \neq 0 \wedge x \mapsto e' \star \mathrm{list}(e')\}$$
$$t := *x;$$
$$\{x \neq 0 \wedge x \mapsto e' \wedge t = e' \star \mathrm{list}(e')\}$$